STRUCTURED TECHNICAL REPORT

SOC Investigation #1

LAB-2026-001: Encoded PowerShell, a Temporary Marker File, and an Internal TCP Connection

Triage | Process Correlation | Packet Evidence | Response | Closure

PREPARED BY	Jackson Blase

REPORT DATE	November 10, 2026
PROJECT PERIOD	September 2026

Table of Contents

Bottom Line Up Front (BLUF) **4**

1. Case Profile, Objectives, and Authority **4**
- 1.1 Case Profile 4
- 1.2 Objectives 5
- 1.3 Authorized Safety Boundary 5
- 1.4 Initial Hypotheses 5

2. Preparation and Baseline **5**
- 2.1 Validate Clocks and Sensor Health 5
- 2.2 Verify Both Search Paths 6
- 2.3 Establish Packet Visibility 6
- 2.4 Endpoint-Protection Limitation 6

3. Unexpected Baseline Alert **7**
- 3.1 Why the Exercise Paused 7
- 3.2 Process GUID Pivot and Raw XML 7
- 3.3 Decode the Baseline Command 7
- 3.4 Baseline Disposition 8

4. Controlled Activity and Start Boundary **8**

5. Endpoint Correlation and Command Analysis **8**
- 5.1 Identify the Controlled Process 8
- 5.2 Extract Fields from XML and Constrain the Search 9
- 5.3 Decode Safely 10

6. File Effects and Wazuh Alert Assessment **10**
- 6.1 The Intended Marker File 10
- 6.2 The Two Wazuh Alerts 10

7. Packet Analysis **11**
- 7.1 Preserve and Verify the Capture 11
- 7.2 The Six-Packet Exchange 11
- 7.3 Interpretation and Limits 12

8. Scope and Evidence Preservation **12**
- 8.1 Separate Search Mentions from Actual Actions 12
- 8.2 Preserve Before Cleanup 13

9. Response, Cleanup, and Recovery **13**
- 9.1 Containment Decision 13
- 9.2 Targeted Artifact Cleanup 13
- 9.3 Recovery Checks 14
- 9.4 Closure and Recovery Point 14

10. Chronological Timeline **14**

11. Assessment, ATT&CK Mapping, and Limitations **16**
- 11.1 Final Assessment 16
- 11.2 MITRE ATT&CK Mapping 16
- 11.3 Limitations Retained at Closure 16

12. NIST Principles Applied **17**

13. Lessons Learned and Future Investigations **18**
13.1 Lessons Learned 18
13.2 Proposed Detection Improvements (Not Implemented) 18
14. Conclusion **19**
Appendix A. Evidence Register **20**
Appendix B. Sources and References **20**
Glossary **22**

Bottom Line Up Front (BLUF)

LAB-2026-001 was closed as an authorized, benign training case with a final severity of **Informational**. No security incident was identified within the reviewed lab scope. The investigation correlated an encoded PowerShell process, a 100-byte text marker file, and one internal TCP connection using Sysmon, Splunk, Wazuh, file metadata, hashes, and Wireshark packet evidence.

Wazuh rule 92057 correctly detected encoded PowerShell execution: a true behavioral positive with an authorized, benign explanation. Rule 92213 classified a short-lived PowerShell policy-test script as an executable file drop mapped to T1105, which was a detection false positive in this case. The controlled network test completed a TCP handshake and orderly close on port 443 with zero bytes of application payload; it did not establish a TLS or HTTPS session.

Evidence was preserved before the marker file was removed, and `MarkerExistsAfterCleanup=False` verified the cleanup. Post-cleanup searches showed current Sysmon ingestion in Splunk and recent endpoint alerts in Wazuh. No isolation, process termination, security-control change, or broad deletion was justified.

The case was conducted on September 28, 2026. This report was prepared on November 10, 2026, from the contemporaneous case record, the notes recorded during the case, and the original packet capture, whose SHA-256 hash still matches the September value. [S1–S3] The report closes with lessons learned and proposed detection improvements.

1. Case Profile, Objectives, and Authority

1.1 Case Profile

Case ID	LAB-2026-001
Scenario	Suspicious encoded PowerShell, temporary-file creation, and an internal TCP/443 connection
Analyst and Owner	Jackson Blase
Environment	Authorized personal cybersecurity lab
Endpoint	WORKSTATION-01; Wazuh agent SOC-Windows-01, ID 001
User	`WORKSTATION-01\analyst`
Monitoring Server	`wazuh-server`, 192.168.56.10
Initial Priority	Medium / P3 (training label); impact not yet established
Final Severity and Status	Informational; closed as authorized lab activity
Confidence	High for the reviewed event chain, subject to the limitations in Section 11.3

P3 is the case's training priority, not a NIST or Wazuh standard. Wazuh rule levels and analyst-assigned incident severity are different measures.

1.2 Objectives

1. Validate authorization, telemetry, clock accuracy, and the evidence-collection path.
2. Detect suspicious-looking execution and assess the credibility of the alerts.
3. Preserve source details before making any changes.
4. Link the process to its parent, user, files, and network activity.
5. Decode the Base64 command safely as text.
6. Determine scope and impact, and whether the observations meet incident criteria.
7. Validate the automated rule descriptions and MITRE ATT&CK mappings.
8. Select a proportionate response, clean up the test artifact, verify monitoring, and document lessons learned.

1.3 Authorized Safety Boundary

The controlled activity was limited to a readable encoded PowerShell script, one plain-text marker file in the current user's Temp directory, and a TCP connectivity test to the owned internal server at 192.168.56.10:443. Remote payload retrieval, persistence, credential access, weakening of security controls, exploitation, and public-internet targets were all outside the test.

The Windows host served as both the monitored endpoint and the analyst workstation. The observed traffic used a VirtualBox host-only link, and the VM's separate NAT adapter remained available for updates. Because the environment was not air-gapped, scope conclusions apply to the available lab telemetry, not to an enterprise fleet.

1.4 Initial Hypotheses

Hypothesis	Evidence Needed	Final Assessment
Authorized Administrative or Lab Activity	Known user and test time, benign decoded logic, expected artifact and destination	Confirmed
Malicious PowerShell or Staging	Suspicious parent, downloaded payload, persistence, credential access, external destination, unexpected spread	Not supported by the reviewed command and effects
Ordinary Software Triggering a Broad Detection	Legitimate context, short-lived runtime artifact, mismatch between rule wording and raw event	Confirmed for rule 92213; rule 92057 still correctly identified the encoded behavior

2. Preparation and Baseline

2.1 Validate Clocks and Sensor Health

On the case date, Windows displayed local time in Central Daylight Time (UTC−05:00), and Ubuntu used UTC with NTP enabled and synchronized. Windows names the zone "Central Standard Time" year-round; that label does not mean a UTC−06:00 offset was in effect on September 28. Both the original timestamps and their effective offsets were retained.

The first query for recent Sysmon Event ID 1 records returned nothing in its narrow time window. A follow-up log-metadata query failed with *Attempted to perform an unauthorized operation*. A later "log not found" message resulted from an empty variable left by that failure; it was not proof that the channel was missing. Repeating the read-only inspection from an Administrator PowerShell session confirmed:

- Sysmon64 running, with Automatic startup.
- The Sysmon Operational log enabled.
- 56,470 records, with circular retention.
- Recent Event IDs 1, 17, and 18 at approximately 16:35:21 local time.

```
Get-Service -Name Sysmon64
Get-WinEvent -ListLog "Microsoft-Windows-Sysmon/Operational" |
  Select-Object LogName, IsEnabled, RecordCount, LogMode
Get-WinEvent -LogName "Microsoft-Windows-Sysmon/Operational" -MaxEvents 10 |
  Select-Object TimeCreated, Id, ProviderName
```

These checks separately establish that the service is running, the log exists, events are current, and the caller has sufficient access rights. Circular retention also means indexed copies and exports matter, because older local events are eventually overwritten.

2.2 Verify Both Search Paths

Splunk returned 11,104 Sysmon events from the preceding 15 minutes, the newest at 16:37:23.874, from the expected Windows Sysmon source and XML sourcetype. Wazuh showed SOC-Windows-01 as Active, with 1,015 alerts in 24 hours and 213 in the most recent 15 minutes. These results established collection health before the controlled test. High counts showed activity and potential noise, not maliciousness.

2.3 Establish Packet Visibility

Wireshark captured on Ethernet 2, the VirtualBox host-only adapter. The early view showed normal Wazuh agent traffic on TCP 1514. The display filter was then narrowed to:

```
arp || (ip.addr == 192.168.56.10 && tcp.port == 443)
```

Before the controlled test, the narrowed view showed ARP traffic and no TCP/443 traffic, which provided a clean baseline. A display filter changes which packets are shown, not which packets are retained in the PCAP.

2.4 Endpoint-Protection Limitation

Malwarebytes Detection History could not be opened because its Tamper Protection password was unavailable. The application was not bypassed, weakened, or reinstalled during the case, so its detection-history result is unknown. Other independent sources remained available, and the gap was recorded rather than treated as a finding of no detections.

After the case, the Tamper Protection password was recovered. Malwarebytes then confirmed the information the case needed, and all of its settings were verified as configured.

3. Unexpected Baseline Alert

3.1 Why the Exercise Paused

At approximately 16:40, Wazuh showed a pre-existing level-15 alert from rule 92213, *Executable file dropped in folder commonly used by malware*. Because the alert preceded the planned exercise, test activity was paused so that a possible existing incident would not be mixed with new simulated activity. The underlying Sysmon Event ID 11 record showed:

Creating Image	`C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe`
User	`WORKSTATION-01\analyst`
PID / Process GUID	37740 / `{ddd355be-dee7-6aba-a564-010000001b00}`
Target File	`C:\Users\analyst\AppData\Local\Temp\__PSScriptPolicyTest_meupsgmq.cpm.ps1`
Embedded Event Time	2026-09-28 21:40:55.291Z (16:40:55.291 local)
Alert Time	2026-09-28 16:40:53.416 −05:00

The filename suggested PowerShell's normal policy-evaluation behavior, but a plausible filename alone was not sufficient for a benign classification.

3.2 Process GUID Pivot and Raw XML

An exact Process GUID search in Splunk returned 12 related Sysmon records spanning about 1.3 seconds. The first results table showed `_time` while the other requested columns were empty, which indicated missing field extraction rather than missing evidence; opening the raw XML revealed the data. An accidental click on `_raw` added an overly specific escaped filter that returned zero results. The search was reset rather than treating that empty result as a new finding.

Sysmon Event ID 1 identified the parent process as `ChatGPT.exe` (PID 6276), belonging to the installed Codex desktop application, and the child as non-interactive Windows PowerShell launched with `-EncodedCommand`. File-deletion records showed that the policy-test files were removed within milliseconds. Named-pipe creation and connection events (Event IDs 17 and 18) were also present; their presence alone did not indicate compromise.

3.3 Decode the Baseline Command

The encoded value was decoded from Base64 as UTF-16LE text without evaluating or executing it. The command created a `SoundPlayer` object for the locally installed resource `app\resources\codex-notification.wav` and called `PlaySync()`:

```
[System.Media.SoundPlayer]::new(
  'C:\Program Files\WindowsApps\OpenAI.Codex_26.924.1866.0_x64__2p2nqsd0c76g0\app\resources\codex-
notification.wav'
).PlaySync()
```

The command plays a local notification sound; it performs no download, persistence, credential access, or network activity. The expected parent process, the exact decoded logic, the files' short lifetime, and the deletion telemetry all converged on a benign explanation.

In the Event ID 26 (file deletion) record, the field `IsExecutable=false` indicated that Sysmon did not identify the file as a PE executable. It does not establish that a `.ps1` script is harmless or cannot run; the false-positive assessment rests on the full context, not on that one field.

3.4 Baseline Disposition

The baseline activity was legitimate local tooling. Rule 92213's implication of an executable drop and ingress tool transfer (T1105) was unsupported for this policy-test artifact. Severity was reduced to Informational, and isolating the host or deleting legitimate application components was not justified. Repetitive shell alerts were noted as a separate tuning issue; one decoded helper command did not explain the origin of every later shell alert.

4. Controlled Activity and Start Boundary

The test start was recorded at 17:27:22.517 −05:00 (2026-09-28T22:27:22.5171916Z). The intended marker file was `C:\Users\analyst\AppData\Local\Temp\LAB-2026-001-marker.txt`. The controlled `-EncodedCommand` decoded to:

```
$caseId = 'LAB-2026-001'
$marker = Join-Path $env:TEMP 'LAB-2026-001-marker.txt'
@(
  "CaseId=$caseId"
  "Timestamp=$((Get-Date).ToString('o'))"
  "Host=$env:COMPUTERNAME"
  "User=$env:USERNAME"
) | Set-Content -LiteralPath $marker -Encoding UTF8
Test-NetConnection -ComputerName 192.168.56.10 -Port 443 -InformationLevel Detailed
```

`Join-Path` builds the user-specific file path, and the array supplies the marker's four lines. The `Set-Content -LiteralPath` cmdlet writes to that exact path, and `-Encoding UTF8` selects the text encoding. `Test-NetConnection` tests TCP reachability to the specified server and port; it does not send a web request or download a file.

The test output reported source 192.168.56.1 on Ethernet 2, destination 192.168.56.10:443, `TcpTestSucceeded=True`, and `IsAdmin=False`. The decoded logic was then compared with the observed effects instead of being trusted simply because it was the expected lab script.

5. Endpoint Correlation and Command Analysis

5.1 Identify the Controlled Process

Sysmon Event ID 1 at 17:27:22.531 recorded:

Image	`C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe`
Process ID	33924
Process GUID	`{ddd355be-e9ca-6aba-1e79-010000001b00}`
User / Integrity Level	`WORKSTATION-01\analyst` / Medium

Parent	Interactive `powershell.exe`, PID 4648
Command Line	`-NoLogo -NoProfile -NonInteractive -EncodedCommand` followed by the captured Base64 string
Image SHA-256	`60C9B29843624DD8AF6B6A7B26147753BE6AEEF65098B9A503CC50850AFE410F`

The image hash identifies the PowerShell executable recorded by Sysmon, not the script's contents; a familiar path or hash says nothing about the command's intent. The parent-child relationship matched the authorized launch from the terminal.

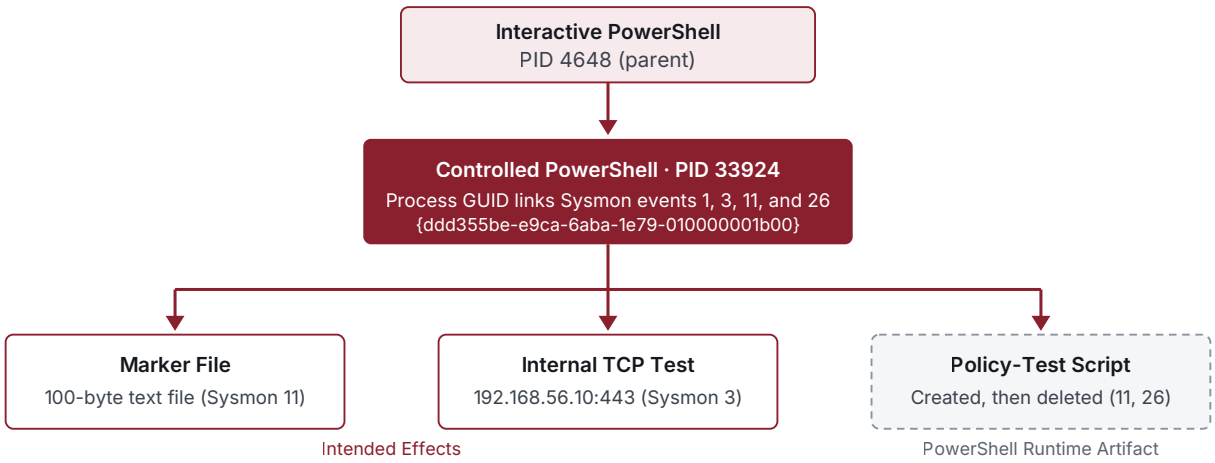


Figure 1. *Controlled process relationships, reconstructed from the case's Sysmon fields. The marker file and internal TCP test were the intended effects; the policy-test script was a separate PowerShell runtime artifact. This is an analytical diagram, not an EDR screenshot. [S1]*

5.2 Extract Fields from XML and Constrain the Search

The original investigation used narrow time bounds, a Process GUID pivot, raw-event review, and `rex` extraction when fields were blank. The archival example below uses the case date instead of a relative "last 24 hours" window, which would no longer cover the September case:

```
index=* sourcetype="XmlWinEventLog:Microsoft-Windows-Sysmon/Operational"
  earliest="09/28/2026:17:27:15" latest="09/28/2026:17:27:40"
  "{ddd355be-e9ca-6aba-1e79-010000001b00}"
| rex field=_raw "<EventID>(?<EventID>\d+)</EventID>"
| rex field=_raw "Name='ProcessGuid'>(?<ProcessGuid>[^<]+)"
| rex field=_raw "Name='ProcessId'>(?<ProcessId>[^<]+)"
| rex field=_raw "Name='Image'>(?<Image>[^<]+)"
| rex field=_raw "Name='CommandLine'>(?<CommandLine>[^<]+)"
| rex field=_raw "Name='TargetFilename'>(?<TargetFilename>[^<]+)"
| rex field=_raw "Name='DestinationIp'>(?<DestinationIp>[^<]+)"
| rex field=_raw "Name='DestinationPort'>(?<DestinationPort>[^<]+)"
| sort 0 _time
| table _time EventID ProcessGuid ProcessId Image CommandLine TargetFilename DestinationIp DestinationPort
```

The `index=*` term searches all non-internal indexes the analyst can access; `sourcetype` narrows the data format; and the time bounds isolate the event chain. Each `rex` command creates a field from a named XML element, `sort 0` sorts all results without the default limit, and `table` selects the output columns. The single-quote pattern matches the XML format observed in this case; other XML serializations may need adjusted patterns. The search head's time zone and permissions should be checked before reusing this example.

A raw GUID match can also occur in a record's parent-process fields. Extracting and checking each event's own `ProcessGuid` prevents every raw text hit from being attributed to the same process. An earlier combined IP search also returned System-process traffic on port 137 with a different GUID; those NetBIOS rows were excluded from this PowerShell connection assessment.

5.3 Decode Safely

```
# Supply only the captured Base64 string, as data.
[Text.Encoding]::Unicode.GetString([Convert]::FromBase64String($encodedCommand))
```

`FromBase64String` converts the text to bytes, and .NET's `Encoding.Unicode` decodes them as UTF-16LE. The result is displayed as text only; it must never be passed to `Invoke-Expression`, a shell, or any other execution step during analysis. The recovered script fully explained the marker file and connectivity test, with no hidden additional operations.

6. File Effects and Wazuh Alert Assessment

6.1 The Intended Marker File

Filesystem metadata recorded the marker's creation at 17:27:23.1435008 −05:00, and Sysmon Event ID 11 recorded it at 17:27:23.150 under the controlled Process GUID. The file contained the case ID, a timestamp, the host name WORKSTATION-01, and the user name analyst. It was 100 bytes, and its text matched the authorized script.

Marker SHA-256	F2A47E3F60EE0AF8F7DEBA270F332F4707F8D08CB8169A9183054C8B0EEE54C1

This hash was collected before deletion. The original marker no longer exists by design; its hash and contents are preserved in the case record.

6.2 The Two Wazuh Alerts

The exact-process query returned two alert documents:

```
agent.name:"SOC-Windows-01" AND
  data.win.eventdata.processGuid:"{ddd355be-e9ca-6aba-1e79-010000001b00}"
```

Alert	Raw Evidence	Assessment
Rule 92057, Level 12, 17:27:22.628	Sysmon record 1324936; Wazuh ID 1790634442.8972346; encoded child PowerShell	True behavioral positive. The decoded script and context establish authorized, benign activity.

Alert	Raw Evidence	Assessment
Rule 92213, Level 15, 17:27:23.616	Sysmon record 1324938; Wazuh ID 1790634443.8982073; target `__PSScriptPolicyTest_0uukxbvw.ko5.ps1`	False positive: the executable-drop and ingress-transfer interpretation does not fit this artifact.

Both records are in the index `wazuh-alerts-4.x-2026.09.28`. Their OpenSearch `_id` values are `Tc0h6qAByNj94JGnSWYM` (92057) and `Ts0j6qAByNj94JGndWZN` (92213). The Wazuh ID and Sysmon `eventRecordID` fields provide additional references to the same evidence. To pivot to the PowerShell process, use `data.win.eventdata.processId` 33924 and its Process GUID; the system header's `processID` 4984 belongs to the event provider's execution context. [S3, S8]

The level-15 alert did not target the case's marker file. It targeted a temporary PowerShell policy-test script created at 17:27:22.619 (22:27:22.619Z) and deleted at about 17:27:22.624, roughly five milliseconds later. The alert's `rule.firedtimes=16` value is a counter stored in the document, not evidence of 16 malicious files in this case.

The Process GUID alert view did not show a corresponding network alert. This is a limitation of the returned alert results, not evidence that the connection was absent or that the agent never collected it. Splunk and packet evidence independently confirmed the network activity.

7. Packet Analysis

7.1 Preserve and Verify the Capture

The capture was saved at approximately 17:29:15 as `04-LAB-2026-001-controlled-powershell.pcapng` (10,697,808 bytes). It contains background host-only traffic as well as the controlled connection. Its whole-file SHA-256, collected during the case and independently rechecked on November 10, is:

Capture SHA-256	`A861C30E16B7601E0C14FF3508ED5DC02CFAE39F86B3836F4F4E2DA09B4D7F03`

An offline display filter isolated one TCP stream:

```
ip.addr == 192.168.56.10 && tcp.port == 443
```

7.2 The Six-Packet Exchange

Frame	Direction	Flags	Time (Local, UTC−05:00)
18366	Windows 192.168.56.1:59225 → Wazuh 192.168.56.10:443	SYN	17:27:26.6320309
18367	Wazuh → Windows	SYN, ACK	17:27:26.6322916
18368	Windows → Wazuh	ACK	17:27:26.6323604
18373	Windows → Wazuh	FIN, ACK	17:27:26.6350377
18376	Wazuh → Windows	FIN, ACK	17:27:26.6360453

Frame	Direction	Flags	Time (Local, UTC−05:00)
18377	Windows → Wazuh	ACK	17:27:26.6360867

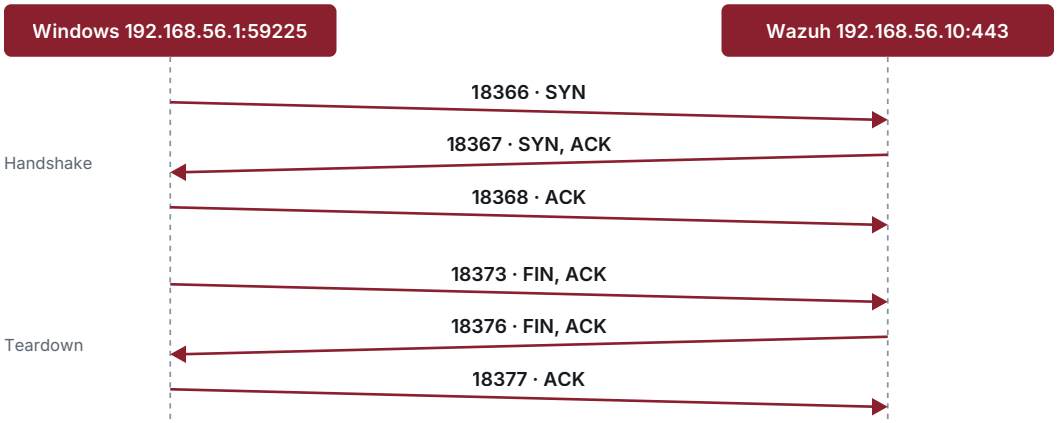


Figure 2. *Packet-derived connection sequence, independently rechecked from the original PCAP. Every segment has `tcp.len=0`. The server combined its acknowledgment of the client's FIN with its own FIN, so the teardown took three segments instead of the classic four. [S2]*

The entire exchange lasted approximately 4.056 milliseconds and is stream 1 in this file. Filtering directly on `tcp.stream == 1` returned the same six frames. Gaps in the frame numbers reflect other traffic in the capture. The contemporaneous Wireshark view reported zero dropped packets. [S2, S3]

7.3 Interpretation and Limits

The evidence proves that the Windows host reached a TCP listener accepting connections at the internal address and port. The selected stream contains no TLS ClientHello, HTTP request, or application payload. It supports a successful TCP reachability test with orderly termination, not an HTTPS transaction or tool transfer.

The endpoint's Sysmon Event ID 3 record is timestamped 17:27:28.115, about 1.483 seconds after the first packet. The process attribution and endpoints agree, but the time difference was not resolved. Logging or collection delay is a possible explanation, not a confirmed cause, and the original times are kept separate in the timeline.

8. Scope and Evidence Preservation

8.1 Separate Search Mentions from Actual Actions

A 24-hour search for the marker filename returned three records on WORKSTATION-01, across three different Process GUIDs. Reviewing their event types and command lines showed:

Time	Event	What It Actually Represented
17:26:18.962	Sysmon 1, PID 37372	The analyst's payload-preparation command, run through the Codex desktop app's PowerShell runtime; the filename appears in the command text

Time	Event	What It Actually Represented
17:27:23.150	Sysmon 11, PID 33924	The single recorded creation of the marker file
17:31:52.208	Sysmon 1, PID 36652	The analyst's read-only evidence command hashing the marker and PCAP; the filename appears in the command text

Both Event ID 1 records used Codex's bundled `pwsh.exe` with the sandbox command runner as the parent. They were not additional marker creations. No other host appeared in the marker-name search across the indexed data available to the analyst.

```
index=* earliest="09/27/2026:17:47:00" latest="09/28/2026:17:47:20"
  "LAB-2026-001-marker.txt"
| rex field=_raw "<EventID>(?<EventID>\d+)</EventID>"
| rex field=_raw "Name='ProcessGuid'>(?<ProcessGuid>[^<]+)"
| stats count AS Events earliest(_time) AS FirstSeen latest(_time) AS LastSeen
    values(host) AS Hosts values(EventID) AS EventIDs values(ProcessGuid) AS ProcessGuids
| convert ctime(FirstSeen) ctime(LastSeen)
```

This archival example reproduces the case's approximate 24-hour window. The original investigation also used a detailed chronological table to classify the three hits. A count of matching records cannot be read as a count of executed payloads without that action-level review.

8.2 Preserve Before Cleanup

The marker's metadata, contents, and hash were recorded alongside the capture's metadata and hash:

```
$markerPath = Join-Path $env:TEMP 'LAB-2026-001-marker.txt'
# $capturePath was set to the actual saved PCAP path during the case.
Get-Item -LiteralPath $markerPath, $capturePath |
  Select-Object FullName, Length, CreationTimeUtc, LastWriteTimeUtc
Get-Content -LiteralPath $markerPath
Get-FileHash -LiteralPath $markerPath, $capturePath -Algorithm SHA256
```

The initial output table truncated the displayed hashes with ellipses, so the full SHA-256 values were retrieved and recorded instead of copying the shortened display. Hashes provide an integrity baseline; they do not replace provenance records, collection notes, or a formal chain of custody.

9. Response, Cleanup, and Recovery

9.1 Containment Decision

No host isolation, process termination, permanent block, or security-control change was performed. Known authorization, benign decoded content, limited scope, an expected internal destination, and the observed effects did not justify disruptive containment. This was a documented response decision, not a skipped step.

9.2 Targeted Artifact Cleanup

After the evidence was preserved, the 100-byte marker file was displayed and then removed at approximately 17:49 local time:

```
$markerPath = Join-Path $env:TEMP 'LAB-2026-001-marker.txt'
Get-Item -LiteralPath $markerPath | Select-Object FullName, Length, LastWriteTime
Remove-Item -LiteralPath $markerPath
"MarkerExistsAfterCleanup=$(Test-Path -LiteralPath $markerPath)"
```

The `-LiteralPath` parameter targets the exact named file; no wildcard or recursive deletion was used. The final output was `MarkerExistsAfterCleanup=False`. The original PCAP and case documentation were retained. This was removal of an authorized test artifact; no malware eradication or system rebuild was required.

9.3 Recovery Checks

The original recovery query used a relative five-minute window at the time it was run:

```
index=* earliest=-5m host="WORKSTATION-01"
  sourcetype="XmlWinEventLog:Microsoft-Windows-Sysmon/Operational"
| stats count AS Events latest(_time) AS LatestEvent values(source) AS Sources
| convert ctime(LatestEvent)
```

It returned 3,842 Sysmon events in the 17:45:51–17:50:51 window, the newest at 17:50:49.734. A Wazuh agent-name search over 17:47:08.969–17:52:08.969 returned 85 alerts through wazuh-server, the newest at 17:52:05.642. Each window spanned the cleanup, and the latest timestamps confirmed that collection continued after the marker was removed.

Rules 92032 and 92052, and another rule 92213 finding, appeared in the recovery view. These alerts require their own review; they are not evidence that the marker remained. Rule level alone was not used as a reason to reopen the benign test case.

9.4 Closure and Recovery Point

The case was closed as Informational after the saved evidence and monitoring checks were confirmed. The Wazuh VM was then shut down from within Ubuntu with `sudo poweroff`, and the post-case snapshot was taken once VirtualBox showed it as Powered Off. The VM definition records the snapshot *Post-LAB-2026-001 clean state – 2026-09-28*, created 2026-09-28 23:12:06Z (18:12:06 local). The snapshot preserves a VM recovery point; the PCAP and report are stored separately on the Windows host. [S3]

10. Chronological Timeline

All times are local on September 28, 2026 (UTC−05:00). Times marked "about" are approximate analyst actions. Event time, alert time, and evidence-save time are distinct.

Time	Observation or Action	Significance
About 16:24	Case opened; Windows clock recorded	Authority and time boundary established
About 16:28	Agent Active; Ubuntu UTC and NTP checked	Server-endpoint communication and time basis confirmed
About 16:30–16:35	Sensor checks, access error, elevated log inspection	Sysmon operational; permissions distinguished from health

Time	Observation or Action	Significance
16:37:23.874	Newest of 11,104 Sysmon events in preparation search	Splunk collection current
16:40:53.416	Wazuh baseline rule 92213 alert time	Planned exercise paused
16:40:55.177	Baseline helper process created	Codex desktop parent; encoded child PowerShell
16:40:55.291–.296	Policy-test file created and deleted	Short-lived runtime artifact; clock and event fields preserved separately
About 16:52	Baseline command decoded	Local notification sound explains the helper
About 16:55	Malwarebytes history unavailable	Evidence limitation recorded
17:18:18.481707	First packet in the case capture	Packet collection under way
About 17:19	Display narrowed to ARP and TCP 443	No test connection visible before execution
17:26:18.962	Payload preparation and encoding command	Marker filename mentioned; no file created
17:27:22.517	Authorized test start recorded	Exact controlled boundary
17:27:22.531	Sysmon 1, PID 33924	Controlled process identity
17:27:22.619–.624	Controlled process's policy-test script created and deleted	Separate from the marker; five-millisecond lifetime
17:27:22.628	Wazuh rule 92057 document timestamp	Encoded PowerShell detection
17:27:23.1435008	Marker filesystem creation time	Intended text-file effect
17:27:23.150	Sysmon 11 marker record	Process attribution
17:27:23.616	Wazuh rule 92213 document timestamp	Policy-test artifact misclassified
17:27:26.6320309–.6360867	Six-packet TCP stream	Internal reachability; zero payload
17:27:28.115	Sysmon 3 connection record	Same process and endpoints; timing differs from PCAP
17:28:50.768794	Last packet in the capture	Capture duration about 632.287 seconds
About 17:29:15	Capture saved	Durable packet evidence
17:31:52.208	Evidence-hashing command	Second filename mention, not another creation
About 17:39	Controlled command decoded and reviewed	Recovered text matches file and network effects
About 17:49	Marker removed; absence verified	Authorized cleanup complete
17:50:49.734	Latest Sysmon event in recovery search	Splunk ingestion still current
17:52:05.642	Latest Wazuh alert in recovery search	Agent-to-manager alert path still current
At closure	Informational disposition; no incident	Evidence-supported response decision

Time	Observation or Action	Significance
18:12:06	Post-case snapshot recorded	Recoverable VM state after orderly shutdown

11. Assessment, ATT&CK Mapping, and Limitations

11.1 Final Assessment

The observed scope was one endpoint, one user, one marker-file creation, and one internal TCP test. The decoded script contained no persistence, credential access, public destination, remote code retrieval, or security-control changes. The decoded command, parent process, user, marker contents, Process GUID correlation, internal destination, zero-payload packet exchange, scope search, and cleanup result all support authorized, benign activity, and the investigation found no security impact in the chain. Rule severity was reassessed in light of intent, scope, and effect.

Question	Determination
Did Encoded PowerShell Execute?	Yes; rule 92057 correctly identified the behavior.
Was a Malicious Executable or Ingress Tool Transferred?	Not supported. The policy-test script and benign marker explain the files, and the selected stream carried no payload.
Did the Case Involve More than One Endpoint?	No other endpoint appeared in the available marker-name search.
Was Isolation Required?	No; it would have been disproportionate to the established activity.
Was Cleanup Complete?	Yes. The marker is absent, and preserved evidence was retained.
Did Monitoring Continue?	Yes, according to the post-cleanup searches.
Is the Entire Computer Certified Free of Compromise?	No. A full-system assessment was neither performed nor claimed.

11.2 MITRE ATT&CK Mapping

Technique	Evidence-Based Interpretation
T1059.001 – Command and Scripting Interpreter: PowerShell	Behavior confirmed; authorized, benign use. A technique name does not assign intent.
T1027 – Obfuscated Files or Information	Encoding observed; Base64 alone does not establish malicious obfuscation.
T1105 – Ingress Tool Transfer	Wazuh's automatic mapping was unsupported for the short-lived local policy-test artifact; neither the decoded command nor the selected packet stream shows a transfer.

11.3 Limitations Retained at Closure

- Malwarebytes Detection History was unavailable at closure; it was reviewed after the Tamper Protection password was recovered (Section 2.4).

- One baseline Wazuh alert is timestamped about 1.875 seconds before its embedded Sysmon event time; the discrepancy was not resolved (Section 3.1).
- The controlled Sysmon Event ID 3 time differs from the packet time by about 1.483 seconds; the cause was not resolved (Section 7.3).
- The Wazuh query returned alerts, not a complete archive of every collected event. The absence of a displayed network alert does not prove the event was not collected.
- Raw event exports (EVTX or JSON) were not created during the case. The effect of that gap and the factors that offset it are explained below.
- The marker was intentionally deleted after preservation and cannot be independently rehashed.
- As the analyst's first structured investigation, the case followed a learning curve in documentation. Screenshots were taken only where the workflow called for them, and the Wazuh and Splunk screens themselves were not captured. Their contents survive as the field values, record identifiers, counts, and timestamps in the case record, which is why this report presents them as tables and analytical diagrams. Capturing key tool screens at each step is part of the evidence-collection improvement in Section 13.2.

Raw Event Exports. The endpoint findings rest on the Sysmon records as indexed in Splunk and Wazuh, and the field values and record identifiers captured in the case record. No raw exports were created, such as an EVTX copy of the Sysmon log or JSON documents from Splunk and Wazuh. Those exports would have let an independent reviewer hash and re-examine the original records outside the tools that indexed them.

The local Sysmon log cannot fill that gap now. It used circular retention with about 56,470 records (Section 2.1), and the preparation search showed about 11,104 Sysmon events every 15 minutes (Section 2.2). At that rate, the log held roughly 75 minutes of activity before overwriting its oldest records, so the September 28 events were overwritten locally within hours of the case.

Three factors limit the effect of this gap. First, the key records carry stable references, including Sysmon EventRecordIDs, Wazuh alert IDs, OpenSearch document IDs, and the Process GUID, which can be re-queried for as long as the Splunk and Wazuh indexes retain them. Second, independent sources agree: the Sysmon process, file, and network records, the marker's filesystem metadata and hash, and the packet capture, whose SHA-256 still matches its September value, all describe the same activity. Third, no conclusion depends on a single unexported record. Future cases will export raw records at collection time with hashes and acquisition notes (Section 13.2).

12. NIST Principles Applied

NIST SP 800-61 Revision 3, finalized in April 2025, aligns incident-response recommendations with the six functions of the NIST Cybersecurity Framework (CSF) 2.0. The case followed the familiar preparation, detection and analysis, response and recovery, and review activities, mapped here to those functions. This is a learning alignment, not a certification or proof of organizational compliance. [S4]

CSF Function	How the Case Applied It
Govern	Defined the owner, authorization, scope, allowed test effects, and authority for a proportionate response
Identify	Listed the endpoint, server, accounts, destinations, dependencies, and evidence sources
Protect	Kept existing safeguards and recovery points in place; did not weaken controls to obtain data

CSF Function	How the Case Applied It
Detect	Verified sensors and clocks; reviewed alerts, raw endpoint records, packet traffic, and artifacts
Respond	Prioritized the unexpected alert, tested hypotheses, preserved evidence, scoped the chain, and documented the no-containment decision
Recover	Removed the test artifact, verified its absence, and confirmed monitoring after cleanup

The Process GUID, Sysmon record IDs, and packet endpoints illustrate correlation of information from multiple sources (DE.AE-03). The evidence register, hashes, and record of actions illustrate the integrity and provenance practices in RS.AN-06 and RS.AN-07. The cleanup and fresh telemetry searches exercised the verification step of recovery in a controlled scenario. The proposed changes to evidence collection and time correlation support the Improvement category (ID.IM). [S4]

The distinction between event, alert, and incident was central. A process start is an event; a matching detection produces an alert; and the analyst determines whether unauthorized harm or a policy violation meets incident criteria. Here, the evidence supported closing the alert case without declaring an incident.

13. Lessons Learned and Future Investigations

13.1 Lessons Learned

The central lesson of this case was to isolate the specific question each check answered. A running service does not guarantee event ingestion, an alert does not prove an incident, and a familiar filename does not prove safety. The case's remaining lessons form a reusable investigation method:

1. Establish authorization, scope, priority, and expected effects.
2. Verify clocks, sensor operation, collection, and search coverage before interpreting negative results.
3. Preserve the alert and source fields before changing the endpoint.
4. Form competing explanations and identify the evidence that could disprove each one.
5. Correlate stable identifiers, especially Process GUID, and confirm what each field actually means.
6. Decode suspicious text without executing it.
7. Compare the intent in the command with the actual process, file, and packet effects.
8. Scope other mentions, users, hosts, and destinations without confusing text matches with executed actions.
9. Choose a response proportional to the demonstrated risk, and document why.
10. Preserve evidence, clean up only the intended targets, and verify services and monitoring afterward.
11. State confidence and gaps, close or escalate the case, and feed lessons back into preparation.

13.2 Proposed Detection Improvements (Not Implemented)

Area	Proposed Improvement
Rule 92057	Keep enabled; enrich alerts with parent, user, integrity level, decoded content, and destination. Do not blanket-allow encoded PowerShell.

Area	Proposed Improvement
Rule 92213	Test narrow handling of verified policy-test artifacts using exact naming, the expected creating process, command context, and short lifetime. The filename and `IsExecutable=false` alone are insufficient.
Repetitive Shell Alerts	Verify each parent and command pattern before adjusting rules 92032 and 92052; retain raw telemetry and detect deviations.
PowerShell Network Visibility	Consider scoped detections for unapproved destinations or suspicious parent-command combinations rather than alerting on every port-443 connection.
Evidence Collection	Export raw event records with acquisition notes and hashes, and capture screenshots of each key alert, search, and raw event as the case progresses, with sensitive details removed.
Time Correlation	Compare source, provider, collector, and packet timestamps explicitly, and resolve unexplained offsets earlier in the investigation.

14. Conclusion

Case LAB-2026-001 was the first structured investigation in the home lab. It began with an unexpected level-15 alert, tested an authorized PowerShell activity against two monitoring paths and independent packet evidence, and closed as Informational with no security incident. The case exercised the full investigative loop: observe, hypothesize, test, interpret, act, and verify. Its most transferable result is the ability to explain why an alarming alert became an Informational case while preserving the evidence and uncertainty behind that decision.

Appendix A. Evidence Register

ID	Evidence	Provenance and Integrity
E-01	Controlled Sysmon 1 process and parent	Indexed-record observations; image hash recorded; separate raw export not supplied
E-02	Sysmon 11 marker creation	Indexed event plus filesystem contents and metadata in the original case record
E-03	Sysmon 3 internal connection	Indexed record tied to the controlled GUID; independently corroborated by PCAP endpoints
E-04	Baseline policy-test alert and helper command	Wazuh and Splunk source fields, decoded command, and creation and deletion context
E-05	Controlled Wazuh 92057 and 92213 documents	Exact document IDs and Sysmon record IDs listed in Section 6.2
E-06	Original case PCAP	10,697,808 bytes; SHA-256 matched the September record on November 10, 2026
E-07	Original marker file	100 bytes; SHA-256 and contents preserved before authorized deletion
E-08	Malwarebytes Detection History	Unavailable during the case; reviewed after the Tamper Protection password was recovered
E-09	Recovery search results	Splunk (3,842 events) and Wazuh (85 alerts) results with latest timestamps
E-10	Post-case VM snapshot	Name and UTC timestamp independently read from the saved VirtualBox definition

The PCAP now resides in the project folder; the September case record lists its earlier Wireshark folder. The matching hash establishes byte-for-byte identity across that move. This report does not claim a legal forensic chain of custody, and the saved case record is not a raw event export. For publication, the host name and Windows user name are replaced with the generic labels WORKSTATION-01 and analyst; hashes and all other evidence values are unchanged.

Appendix B. Sources and References

[S1] `LAB-2026-001-case-report.md`, the contemporaneous case record in `outputs/SOC-Cases/LAB-2026-001`. Primary source for the chronology, event fields, original hash values, response, and closure.

[S2] `04-LAB-2026-001-controlled-powershell.pcapng`, the original capture. Read offline with TShark and Capinfos, and SHA-256 verified November 10, 2026. Selected packets are reproduced as analytical tables and a diagram, without reconstructed application screens.

[S3] Notes recorded during the case, and the saved `Wazuh-Server.vbox` configuration. The notes support the observed query outputs; the VM configuration verifies the snapshot timestamp.

[S4] NIST: SP 800-61 Rev. 3 (final, April 2025) · Cybersecurity Framework 2.0

[S5] Microsoft Sysmon: Sysmon events · Sysmon configuration files

[S6] MITRE ATT&CK: T1059.001 · T1027 · T1105. Technique mapping is an analyst assessment, not a malware verdict.

[S7] Wazuh event logging · Wireshark display filters

[S8] Microsoft, Windows Event Log Execution element. Explains the process identifier in the event's system header.

[S9] Microsoft PowerShell documentation for the glossary: powershell.exe command-line options · PowerShell and application control

Glossary

Terms are defined as they are used in this report. Authoritative references are listed in Appendix B.

Alert, Event, and Incident	An event records observed activity. An alert flags activity that matched a detection rule. An incident involves actual or imminent harm to security or a violation of security policy.
Application Payload and `tcp.len`	Application payload is the data carried inside a TCP connection. Wireshark's `tcp.len` field is a segment's TCP payload length in bytes.
Authorized Simulation and Marker File	The exercise used pre-approved activity within defined limits. The marker was a temporary text file created deliberately to verify file-creation telemetry.
Base64, UTF-16LE, and `-EncodedCommand`	Base64 represents binary data as text. Windows PowerShell's `-EncodedCommand` parameter accepts a Base64 string of UTF-16LE-encoded command text.
Baseline and Control	A baseline records expected activity before testing. A control provides a known reference for evaluating the evidence the test produces.
Case Scope and Disposition	Scope defines the systems, time window, and activity reviewed. Disposition is the analyst's documented judgment and closure outcome.
Containment and Recovery	Containment limits the effects or spread of an incident. Recovery restores or verifies normal operation; this case used targeted cleanup and checks that monitoring still worked.
Correlation and Pivot	Correlation links records that share evidence, such as a host, time, or process identifier. A pivot uses a value from one record to find related records.
Detection Rule and Rule Level	A detection rule evaluates events against defined conditions. A Wazuh rule level (0–15) is the severity the rule assigns to a match.
Endpoint and Sensor Health	The endpoint is the monitored Windows workstation. Sensor-health checks verify that telemetry is still reaching the monitoring tools.
Evidence Integrity and Provenance	Integrity checks establish whether evidence has changed. Provenance records where evidence came from and how it was acquired and handled.
Evidence Register	An inventory of case evidence, with details such as source, time, identifying fields, and hashes where available.
False Positive and True Behavioral Positive	A false positive labels an event as a behavior the evidence does not support. A true behavioral positive correctly detects the specified behavior, which can occur during authorized activity.
Hypothesis and Validation	A hypothesis is a testable explanation for the evidence. Validation checks it against independent observations such as process, file, and packet records.
Informational and Medium/P3	Case-specific analyst priorities. Medium/P3 was the initial training priority; Informational was the final priority for the authorized activity.
Integrity Level	A Windows security label on processes and objects that helps enforce access boundaries. Medium is the default for standard, non-elevated user processes.
`IsExecutable`	A field in Sysmon file-deletion events (IDs 23 and 26) indicating whether Sysmon identified the file as a PE executable. Script files require additional context.

MITRE ATT&CK	A knowledge base of adversary tactics and techniques. This report evaluates T1059.001 (PowerShell), T1027 (obfuscation), and T1105 (ingress tool transfer).
NIST CSF 2.0 and SP 800-61 Rev. 3	The Cybersecurity Framework organizes risk management into six functions: Govern, Identify, Protect, Detect, Respond, and Recover. SP 800-61 Rev. 3 provides incident-response recommendations organized around those functions.
Packet Capture and PCAPNG	A packet capture records the traffic seen on a selected interface. PCAPNG is the file format used to preserve the packets and their metadata.
Policy-Test Script	A short-lived `__PSScriptPolicyTest_` file that PowerShell creates in the Temp folder to check whether an application-control policy (AppLocker or App Control for Business) is enforced. It was a separate file from the case marker.
PowerShell and Script Block Logging	PowerShell is a command shell and scripting language. When enabled, script block logging records processed script content as Event ID 4104 in the Microsoft-Windows-PowerShell/Operational log; it was not a source in this case.
Process ID, ProcessGuid, and ParentProcessGuid	Windows can reuse numeric process IDs. Sysmon's ProcessGuid uniquely identifies a process instance across records, and ParentProcessGuid identifies the process that created it.
Process Image, Command Line, and Parent	The image is the executable's path. The command line records how it was invoked, including arguments. The parent is the process that created the child.
Record ID, Wazuh ID, and OpenSearch `_id`	EventRecordID identifies a record within a Windows event log. The Wazuh ID identifies an alert. The OpenSearch `_id` identifies a document within its index.
`rule.firedtimes`	A Wazuh counter of how many times a rule has fired. It cannot establish how many files or processes belonged to the case.
SHA-256	A cryptographic hash function that produces a 256-bit digest. Matching SHA-256 values show that a file's bytes have not changed.
Snapshot	A saved virtual-machine state used as a recovery point. The report records the Wazuh VM's post-case snapshot.
SOC and Triage	A security operations center monitors and investigates security activity. Triage is the initial review that establishes scope, urgency, and the next evidence to collect.
Splunk SPL, `rex`, `_raw`, and `_time`	Search Processing Language queries events. The `rex` command extracts fields with a regular expression. The `_raw` field holds the original event data, and `_time` is Splunk's assigned event timestamp.
Sysmon Event IDs 1, 3, 11, 17, 18, and 26	Process creation, network connection, file creation or overwrite, named-pipe creation, named-pipe connection, and file deletion detected (logged without archiving the deleted file).
System Header ProcessID	The ProcessID in a Windows event's system header identifies the process that generated the event. Sysmon's EventData ProcessId identifies the process the event describes.
TCP Handshake, Flags, and Teardown	SYN, SYN-ACK, and ACK establish a TCP connection. FIN signals that a sender has finished sending, and each FIN is acknowledged. RST abruptly resets a connection.
TCP Stream and Ephemeral Port	A TCP stream is the set of packets belonging to one connection. An ephemeral port is a temporary source port assigned to the initiating client.
Test-NetConnection	A PowerShell cmdlet that tests TCP reachability to an address and port. The packet capture verified the exchange for this run.

TLS, HTTPS, and TCP/443	TLS establishes an encrypted, authenticated application channel. HTTPS is HTTP over TLS, and 443 is its conventional port. Identifying the protocol requires evidence above the TCP layer.
UTC, Local Offset, and Event Time	UTC is the common timeline reference; local times in this case use UTC−05:00. Event occurrence, collection, and packet timestamps describe different stages or observations.
Wireshark Display Filter	An expression that narrows the packets shown from a capture. Filtering to one TCP stream lets the analyst examine a single exchange in context.